

SGLang 前缀缓存测量与请求流程源码阅读

第二次挑战（HW2） · SGLang v0.5.14 · Qwen/Qwen3-0.6B · 报告正文不超过 8 页（本册 6 页）

0. 实验环境与版本

项目	取值 / 说明
硬件	CPU 推理环境（x86_64，cgroup 限额 4 核 / 8 GB 内存）， 无 GPU ；attention backend 自动回退为 torch_native
推理框架	SGLang 0.5.14（SGLANG_USE_CPU_ENGINE=1 启用 CPU 后端）
Ray	2.56.0（独立于 SGLang 的 Python 环境 /opt/sgl-venv，通过 HTTP 与 SGLang 通信）
模型	Qwen/Qwen3-0.6B（本地权重 /workspace/models/Qwen3-0.6B）
运行时	Python 3.11.1 / torch 2.14.1+cpu / transformers 5.8.1 / Ubuntu 24.04
服务启动	python3 -m sglang.launch_server --model-path /workspace/models/Qwen3-0.6B --device cpu --mem-fraction-static 0.045 --max-total-tokens 16384 --max-running-requests 16；日志确认 Tree cache initialized: impl=RadixCache，前缀缓存处于开启状态

1. 任务一：测量前缀缓存

1.1 实验设计

构造两组负载，除前缀能否复用之外，模型、输入长度、输出长度、请求顺序与并发数完全一致：

- 共享前缀组（shared_prefix）**：32 条请求共享同一段 896 token 前缀，各带 128 token 独立后缀，输入共 1024 token。
- 分散前缀组（dispersed_prefix）**：32 条请求的前缀两两不同（由不同语料排列生成），输入同样为 1024 token。

统一采样参数：temperature=0、max_new_tokens=16、ignore_eos=true、sampling_seed=2026；接口为原生 POST /generate，直接提交 input_ids，并使用流式响应（stream=true）；最大并发 8。

每组执行流程（脚本 src/target1/run_benchmark.py）：① 首次实验前用 2 条 32-token 短请求完成服务预热；② 等待已有请求结束，反复调用 POST /flush_cache 直到返回 Cache flushed. 确认成功；③ 共享前缀组额外发送一条仅含共享前缀的预热请求，使其进入 RadixCache（不计入结果）；④ 以信号量限流并发 8 回放 32 条测量请求。两组各重复 3 轮（run-1/2/3）。

指标定义：**成功率**=HTTP 200 且完整返回的比例；**缓存命中率**=Σcached_tokens / Σprompt_tokens（cached_tokens 取自流式末帧 meta_info）；**实际执行 Prefill 的 token 数**=Σ(prompt_tokens - cached_tokens)；**TTFT**=发出请求到收到首个流式 chunk；**TPOT**=(E2E - TTFT)/(completion_tokens - 1)；**E2E**=发出到收到末帧；p50/p95 采用线性插值百分位。

1.2 对照主表（三轮 run-1/2/3 的平均）

对照主表（三轮 run-1/2/3 的平均）

指标	共享前缀组	分散前缀组	共享 vs 分散
成功率	100.0%	100.0%	—
缓存命中率	89.79%	0.56%	+89.23 pp
实际执行 Prefill 的 token 数（合计）	3,344	32,583	-89.7%
单请求平均实际 Prefill token	104.5	1,018.2	-89.7%
吞吐量（输出 token/s）	8.259	4.618	1.79×

吞吐量 (请求/s)	0.516	0.289	1.79×
TTFT p50 (s)	6.458	18.423	-64.9%
TTFT p95 (s)	6.743	19.185	-64.9%
TPOT p50 (s)	0.599	0.606	-1.1%
TPOT p95 (s)	0.637	0.645	-1.2%
端到端延迟 p50 (s)	15.482	27.682	-44.1%
端到端延迟 p95 (s)	15.643	28.332	-44.8%

「共享 vs 分散」一列：TTFT / TPOT / 端到端延迟与 Prefill token 数为相对变化（负值 = 共享前缀组更低），吞吐为倍数，缓存命中率用百分点（pp）表示。两组 32 条请求均全部成功（成功率 100%），输入长度 1024、输出长度 16 完全一致；共享前缀组命中率更高、实际 Prefill token 更少，满足完成标准。

注：共享前缀组单请求平均命中 919.5 token，略多于设计的前缀长度 896。原因是 RadixCache 缓存的是 请求完整序列——先完成的请求会把它的「896 前缀 + 128 后缀」整条写入 radix tree，后续请求的 128 token 后缀 若与某条已缓存序列的后缀开头偶然重合，最长前缀匹配会再向前延伸到 896 之后（实测单请求命中区间 896–1011 token）。这不是测量误差，而是 radix tree 的真实行为；分散前缀组同理存在 0–27 token 的偶然重合，命中率仅 0.56%，不影响结论。

1.3 为什么相同前缀可以复用 K/V：从因果自注意力出发

$$\text{Attention}(Q, K, V) = \text{softmax}\left(\frac{QK^T}{\sqrt{d_k}}\right)V$$

$$A = \text{softmax}(QK^T / \sqrt{d_k} + M)V, \quad M_{ij} = 0 \ (j \leq i), \quad -\infty \ (j > i)$$

因果掩码 M 使得第 i 个位置的注意力输出只依赖于 x₁..x_i，而不依赖其后的任何 token。又因为 K、V 是逐位置产生的：

$$k_i = f(x_i, \text{pos}(i)), \ v_i = g(x_i, \text{pos}(i)) \Rightarrow K_{1:m}, V_{1:m} \text{ depend only on } x_{1:m}$$

所以当两个请求的前 m 个 token 及其位置完全相同（本实验共享前缀 896 token，位置均从 0 开始对齐，RoPE 的相对位置也一致），它们的 K_{1:m}、V_{1:m} 逐元素相同，RadixCache 只要把这 m 个 KV 索引直接交给新请求即可，无需重算。

命中后仍需计算哪些 token：① 未命中的 n-m 个 token 的 K 与 V（需要写入缓存）；② 这 n-m 个 token 的 Q（它们必须 attend 到包括命中部分在内的完整前缀）；③ 末位的 logits 与第一个输出 token。而已命中的前 m 个 token 的 K、V、Q 全部免算：

$$N_{\text{prefill}} = n - m, \quad \text{per decode step: reads } n \text{ K/V (independent of } m)$$

1.4 TTFT 与 TPOT 为什么呈现不同变化

TTFT 由排队等待 + 本轮 Prefill 计算构成，而 Prefill 的计算量与注意力/MLP 的 token 数成正比，即与 $n-m$ 成正比。共享前缀组 $m \approx 896$ ，实际 Prefill 从 32,583 token 降到 3,344 token（-89.7%），因此 TTFT p50 由 18.423 s 降到 6.458 s（-64.9%）。降幅小于 Prefill token 的降幅，是因为 TTFT 中还包含并发 8 下的排队与调度开销这一"固定成本"。

TPOT 取决于 Decode 每一步的工作量：每步只有 1 个 query，但要读取该序列全部 n 个位置的 K/V。命中只免去 Prefill 的重算，不改变 n ，也不改变每步读 K/V 的量，因此 TPOT 几乎不动：实测 $0.606\text{ s} \rightarrow 0.599\text{ s}$ （-1.1%，属噪声量级）。这正解释了"TTFT 大幅下降、TPOT 基本不变"的现象，也说明前缀复用是一项只优化 Prefill、不优化 Decode 的技术。

端到端延迟 p50 由 27.682 s 降至 15.482 s（-44.1%），输出吞吐由 4.62 提升到 8.26 token/s（约 1.79 $\times$ ），均由 Prefill 段节省的时间直接转化而来。

逐轮明细 (run-1/2/3)

指标	共享 run-1	共享 run-2	共享 run-3	分散 run-1	分散 run-2	分散 run-3
成功率	100.0%	100.0%	100.0%	100.0%	100.0%	100.0%
缓存命中率	89.79%	89.79%	89.79%	0.56%	0.56%	0.56%
实际执行 Prefill 的 token 数（合计）	3,344	3,344	3,344	32,583	32,583	32,583
单请求平均实际 Prefill token	104.5	104.5	104.5	1,018.2	1,018.2	1,018.2
吞吐量（输出 token/s）	8.340	8.226	8.211	4.589	4.650	4.615
吞吐量（请求/s）	0.521	0.514	0.513	0.287	0.291	0.288
TTFT p50 (s)	6.373	6.582	6.420	18.557	18.145	18.567
TTFT p95 (s)	6.773	6.741	6.715	19.209	19.122	19.224
TPOT p50 (s)	0.594	0.598	0.605	0.606	0.608	0.604
TPOT p95 (s)	0.630	0.638	0.644	0.644	0.646	0.643
端到端延迟 p50 (s)	15.264	15.560	15.621	27.897	27.468	27.681
端到端延迟 p95 (s)	15.685	15.605	15.639	28.309	28.234	28.453

三轮结果高度一致（命中率 89.79% / 0.56% 三轮完全相同），说明测量具有良好的确定性：temperature=0 与固定 sampling_seed=2026 下，同一组请求的缓存行为可稳定复现。

SGLang v0.5.14 • 一条 /generate 请求的主流程 (源码级)

主线: TokenizerManager.generate_request → Scheduler.event_loop_normal / waiting_queue → get_new_batch_prefill / match_prefix → PrefillAdder / ScheduleBatch → model worker (Prefill, Decode) → RadixCache

请求接收 • 入队 • 调度

前缀匹配 • 执行 • 缓存写回 • 流式输出



3. 任务二：一条 /generate 请求在 v0.5.14 中的主流程（说明）

3.1 关键函数与文件位置

环节	函数	文件：位置
HTTP 入口	generate_request	srt/entrypoints/http_server.py:769
分词与请求封装	TokenizerManager.generate_request	srt/managers/tokenizer_manager.py:576
主循环	Scheduler.event_loop_normal	srt/managers/scheduler.py:1505
收包与分发	process_input_requests	srt/managers/scheduler.py:1628
入队	_add_request_to_queue → waiting_queue.append	srt/managers/scheduler.py:2258 / 2265
组 prefill batch	get_new_batch_prefill / _get_new_batch_prefill_raw	srt/managers/scheduler.py:2702 / 2722
刷新待算输入	Req.init_next_round_input	srt/managers/schedule_batch.py:1123
前缀匹配	RadixCache.match_prefix (match_prefix_for_req)	srt/mem_cache/radix_cache.py:358 (srt/managers/schedule_policy.py:85)
削减本轮计算量	Req.set_extend_input_len(input_len - len(prefix_indices))	调用 srt/managers/schedule_batch.py:1221 ; 定义 :1525
预算准入	PrefillAdder.add_one_req → can_run_list	srt/managers/schedule_policy.py:858
建 batch / 执行	ScheduleBatch.init_new / run_batch	调用 srt/managers/scheduler.py:2907 (定义 schedule_batch.py:1843) / srt/managers/scheduler.py:3145
模型前向	forward_batch_generation → ModelRunner.forward → _forward_raw	srt/managers/tp_worker.py:65 / srt/model_executor/model_runner.py:2896 / 2987
结果处理	process_batch_result	srt/managers/scheduler.py:3367
完成写回	release_kv_cache → RadixCache.cache_finished_req	srt/mem_cache/common.py:629 → srt/mem_cache/radix_cache.py:438
未完成写回	maybe_cache_unfinished_req → cache_unfinished_req	srt/mem_cache/common.py:113 → srt/mem_cache/radix_cache.py:485
流式输出	handle_loop → _handle_batch_output → _wait_one_response	srt/managers/tokenizer_manager.py:1824 / 1839 / 1425
流式粒度	stream_interval 判定	srt/managers/scheduler_components/output_streamer.py:324

3.2 四个问题的回答

① 请求如何进入等待队列。HTTP 层 http_server.py:769 收到 /generate 后进入 TokenizerManager.generate_request (tokenizer_manager.py:576)：先做参数归一化，再把文本/ids 分词成 input_ids，随后 _send_one_request 经 ZMQ IPC 把 TokenizedGenerateReqInput 发给 Scheduler 进程。Scheduler 在 event_loop_normal (scheduler.py:1505) 每

一轮先 `recv_requests()`，再由 `process_input_requests` (1628) 分发，最终 `_add_request_to_queue` (2258) 把请求 `append` 进 `waiting_queue` (2265) 并记录入队时刻；此后请求才参与组 `batch`。

② **前缀匹配如何减少本轮 Prefill token。** `get_new_batch_prefill` (2702) 遍历 `waiting_queue` 时，先调用 `req.init_next_round_input(self.tree_cache)` (2847)，其内部 (`schedule_batch.py:1166`) 调用 `RadixCache.match_prefix` (`radix_cache.py:358`) 在 `radix tree` 上做最长前缀匹配 (key 先按 `page_size` 对齐)，返回 `prefix_indices` ——即已驻留的 KV cache 槽位索引；随后 `set_extend_input_len(input_len - len(prefix_indices))` (`schedule_batch.py:1221`) 把本轮真正需要计算的 token 数收缩到未命中的那一段。之后 `PrefillAdder.add_one_req` (`schedule_policy.py:858`) 按 `extend_input_len + max_new_tokens + page_size` 校验剩余 token/内存预算，通过者进入 `can_run_list`，由 `ScheduleBatch.init_new` 成批。本实验中共享前缀组因此把合计 Prefill 从 32,583 token 压到 3,344 token。

③ **新产生的 KV 如何写回缓存。** Prefill/Decode 在 `ModelRunner._forward_raw` (`model_runner.py:2987`) 中执行，新算出的 K/V 写入 `token_to_kv_pool_allocator` 分配的槽位，并通过 `req_to_token` 建立 请求→token→槽位的映射。批次返回后 `process_batch_result` (3367) 分两种情况处理：请求结束走 `release_kv_cache` (`common.py:629`) → `RadixCache.cache_finished_req` (`radix_cache.py:438`)，以 (`origin_input_ids + output_ids`) 为 key 把 KV 索引插入 `radix tree`，并把与树中已有节点重复的部分 `kv_indices[cache_protected_len : prefix_len]` 以及未对齐的尾部释放；请求未结束 (`chunked prefill` 或发生抢占) 则走 `maybe_cache_unfinished_req` (`common.py:113`) → `cache_unfinished_req` (`radix_cache.py:485`) 先把已算部分落树。插入期间用 `inc_lock_ref / dec_lock_ref` 保护正在被引用的节点不被 LRU 驱逐。

④ **生成结果如何流式返回。** Scheduler 侧按 `stream_interval` (默认 1，即每生成 1 个 token，见 `output_streamer.py:324`) 把增量输出经 IPC 发回 `TokenizerManager`；`handle_loop` (1824) → `_handle_batch_output` (1839) 把增量写入 `rid_to_state[rid].out_list` 并 `set()` 对应 event；`generate_request` 中的 `_wait_one_response` (1425) 被唤醒后逐个 `yield`，HTTP 层以 SSE (`data: {...}`) 逐帧下发给客户端，末帧 `meta_info` 携带 `prompt_tokens / cached_tokens / completion_tokens / e2e_latency` ——本报告的 `cached_tokens` 与命中率正是从这里采集的。

4. 结论与结果索引

在保持输入/输出长度、请求顺序与并发数一致的前提下，开启 RadixCache 的共享前缀负载把实际 Prefill token 降低 89.7%、TTFT p50 降低 64.9%、端到端延迟 p50 降低 44.1%、输出吞吐提升约 1.79×，而 TPOT 基本不变 (-1.1%)，与"前缀复用只削减 Prefill 计算、不改变 Decode 每步读 K/V 量"的理论分析一致。

报告中的表格/数字	对应的原始结果文件
1.2 主表 (三轮均值)	results/target1/summary.json 的 mean 字段，由 src/target1/aggregate.py 汇总
1.4 逐轮明细	results/target1/{shared_prefix,dispersed_prefix}/run-1 2 3/summary.json
逐请求 TTFT/TPOT/cached_tokens	results/target1/{组}/run-N/per_request.jsonl (每行一条请求)
执行日志 (含 flush_cache 确认、预热)	results/target1/{组}/run-N.log